Web Documentation: Frontend & Backend Architecture

1. Overview & Technology Stack

This section details the web application designed to monitor, manage, and control the TurtleBot security robot. The system is built on a modern Full-Stack layered architecture, ensuring real-time data processing, strict stateless security, and a reactive user interface.

Backend Stack

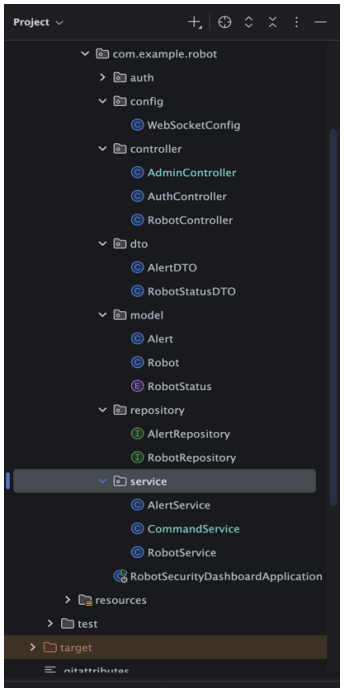
- **Core Framework:** Java 17 with Spring Boot 3.2.3.
- **Architecture Pattern:** Layered MVC (Model-View-Controller) / RESTful API.
- **Security:** Spring Security with JWT (JSON Web Tokens) for stateless, token-based authentication.
- **Database:** MySQL with Spring Data JPA and Hibernate for ORM.
- **Real-Time Communication:** WebSockets using STOMP (Simple Text Oriented Messaging Protocol) and SockJS.

Frontend Stack

- **Core Library:** React.js (built with Vite for optimized bundling).
- **Routing & State:** React Router DOM and React Hooks (useState, useEffect, useRef).
- **HTTP Client:** Axios (configured with request interceptors for automated JWT injection).
- **Real-Time Client:** stompjs and sockjs-client for handling WebSocket subscriptions.

2. Backend Project Architecture

The Spring Boot backend is strictly organized into decoupled layers to separate business logic, data access, and API routing.



- **Controllers (com.example.robot.controller):** Expose the REST API endpoints (RobotController, AdminController). They handle incoming HTTP requests and delegate the work to the service layer.

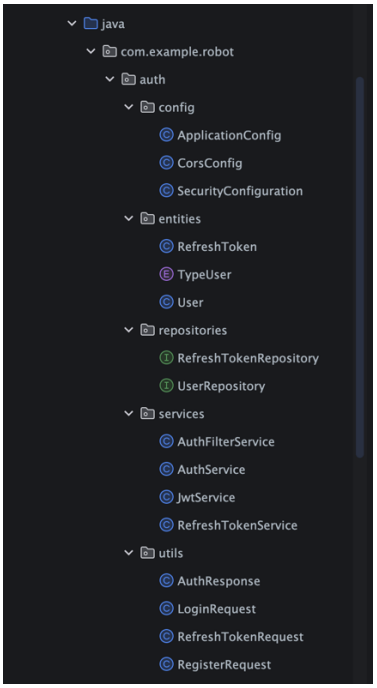
- **Services (com.example.robot.service):** Contain the core business logic. For example, RobotService processes incoming telemetry, checks battery levels, and triggers internal alerts. CommandService acts as an HTTP proxy, forwarding admin commands to the ROS2 robot.

- **Repositories & Models (model, repository):** Manage data persistence using Spring Data JPA interfaces connected to MySQL entities.

- **DTOs (dto):** Data Transfer Objects used to strictly define the shape of incoming JSON payloads, decoupling the API contract from the internal database models.

2.1. The Authentication Module

To maintain high security and clean code, all authentication and authorization logic is isolated within a dedicated auth package.

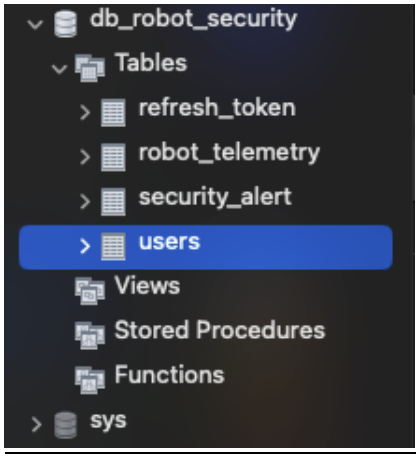


- **Configuration (config):** Defines the SecurityFilterChain, disabling CSRF for stateless APIs, setting up CORS, and configuring route-level permissions (e.g., restricting /api/admin/** exclusively to the ADMIN authority).

- **Services (services):** * JwtService: Handles the generation, parsing, and cryptographic validation of JSON Web Tokens.
    - AuthFilterService: A custom filter extending OncePerRequestFilter that intercepts every HTTP request, extracts the Bearer token, validates it, and loads the user's security context.
    - RefreshTokenService: Manages long-lived sessions to prevent users from having to log in repeatedly.

3. Database Architecture

The application uses a relational MySQL database (db_robot_security). The schema is automatically managed by Hibernate (ddl-auto=update).



- **users**: Stores user credentials. Passwords are encrypted using BCryptPasswordEncoder. It defines the access level via the type column (ADMIN or USER).

- **refresh_token**: Manages extended sessions, storing UUIDs linked to specific users with expiration timestamps.

- **robot_telemetry**: A historical log of the robot's state (X/Y coordinates, battery level, operational status, timestamp).

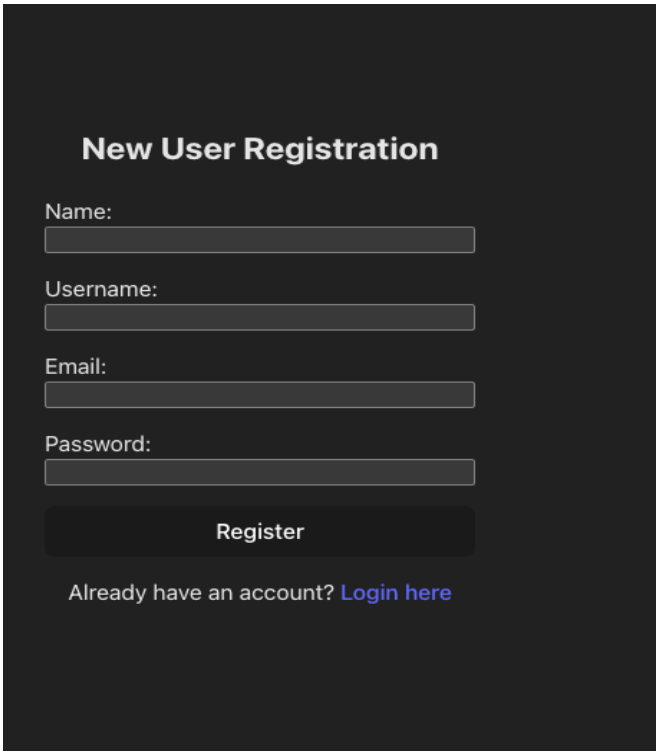
- **security_alert**: Logs security events triggered by the robot's detection module, storing alert types, messages, timestamps, and an intruder snapshot encoded as a Base64 LONGTEXT string.

4. Application Workflows (User vs. Admin)

The system is designed with a seamless onboarding process and implements strict **Role-Based Access Control (RBAC)**. The user journey begins with account creation and diverges significantly based on the assigned role after login.

4.1. Account Registration Flow

Before accessing the dashboard, new personnel must be registered in the system.



1. **User Input:** The user navigates to the registration page and provides their Name, Username, Email, and Password.

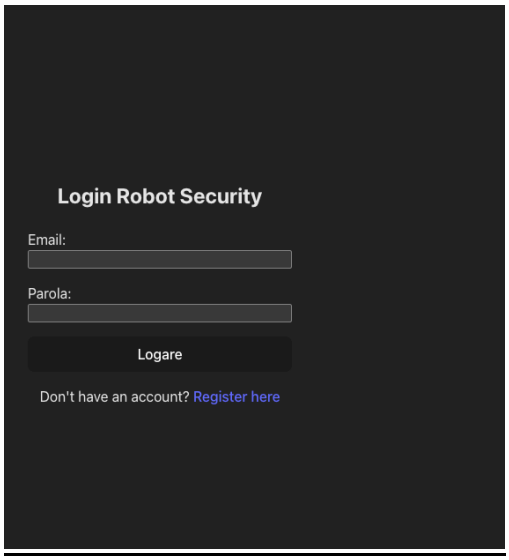
2. **API Request:** Upon form submission, the React frontend sends a POST request with the user's payload to the /v1/auth/register endpoint.

3. **Backend Processing:** The Spring Boot backend intercepts the request. The AuthService extracts the password, encrypts it using BCryptPasswordEncoder to ensure security, and saves the new User entity to the MySQL database.

4. **Completion:** Upon successful database persistence, the backend returns the generated JWT tokens. The frontend displays a success alert and automatically redirects the user back to the Login interface.

4.2. Authentication & The Standard User Flow (Passive Monitoring)

A standard user (Role: USER) is restricted to a read-only monitoring experience to prevent unauthorized interference with the robot.



1. **Authentication:** The user enters their credentials. The React app sends a POST request to /v1/auth/login.

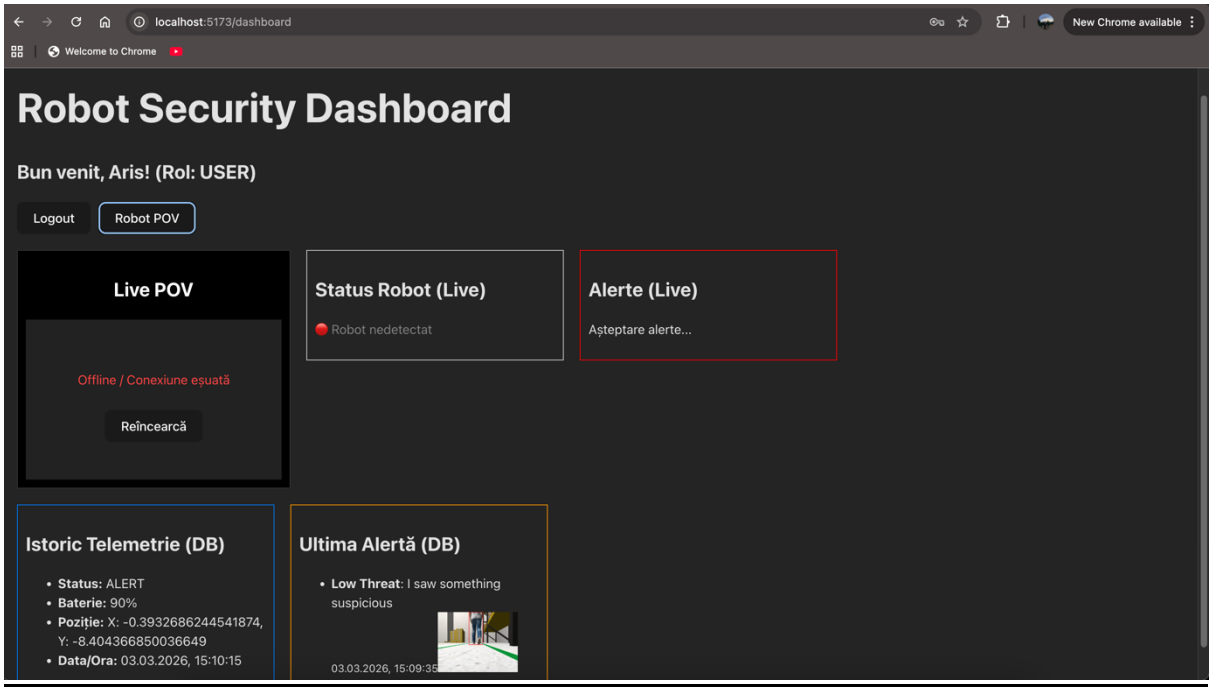
2. **Token Acquisition:** The backend verifies the BCrypt hash against the database. If successful, it returns an Access Token (containing the role claims) and a Refresh Token. React stores these securely in localStorage.

3. **Dashboard Initialization:** React Router navigates to /dashboard. The app sends GET requests to /api/robot/latest-telemetry and /api/robot/latest-alert to load the last known historical state from the database.

4. **WebSocket Connection:** The frontend establishes a SockJS connection to ws://localhost:8080/ws-robot and subscribes to the /topic/telemetry and /topic/alerts channels.

5. **Live Monitoring:** As the robot moves through the warehouse, the UI updates instantly without requiring a page refresh.

6. **Security Restriction:** The Control Panel is completely hidden from the UI. If a user attempts to bypass the UI and send a forged POST request via an external tool (like Postman) to /api/admin/start, the Spring Boot AuthFilterService blocks the request, returning a 403 Forbidden error.



4.3. The Administrator Flow (Active Control)

An administrator (Role: ADMIN) is granted full monitoring capabilities alongside the authority to issue real-time commands to the robot.

1. **Authentication:** Follows the exact same login steps as the Standard User, but the decoded JWT payload now explicitly contains the ADMIN authority.

2. **UI Adaptation:** React detects the ADMIN role from the token and dynamically renders the **Admin Control Panel** alongside the monitoring widgets.

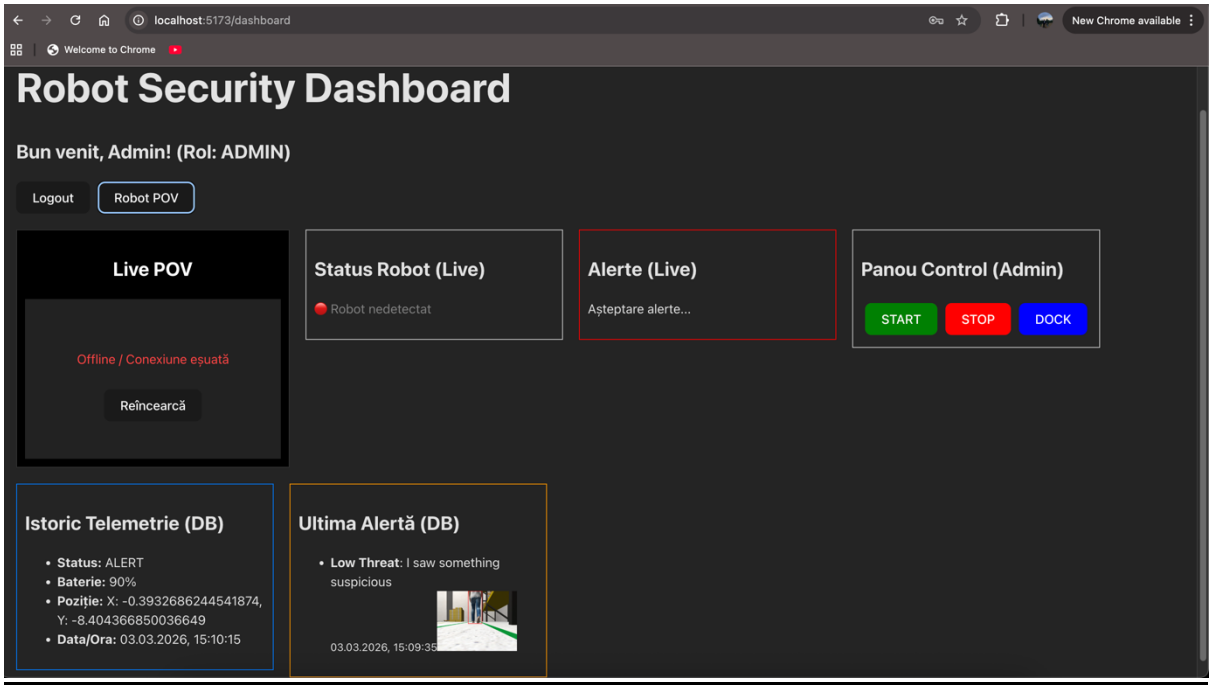
3. **Issuing a Command:** The Admin clicks the START, STOP, or DOCK button on the dashboard.

4. **Secure API Request:** Axios sends a POST request to the corresponding endpoint (e.g., /api/admin/stop) with the Bearer token automatically attached in the headers.

5. **Backend Verification:** Spring Security validates the token and evaluates the @PreAuthorize("hasAuthority('ADMIN')") annotation on the controller.

6. **Command Routing:** The CommandService formats the command into a JSON payload and forwards it via a RestTemplate POST request directly to the ROS2 robot's local Python API (e.g., http://172.20.10.12:5000/api/command).

7. **Robot Execution:** The simulated TurtleBot executes the command in the Gazebo environment and updates its operational status, creating a continuous feedback loop back to the web dashboard via WebSockets.



5. Real-Time Data Flow (Robot -> Backend -> Frontend)

To achieve low-latency updates, the system avoids traditional HTTP polling in favor of a publish-subscribe model.

1. **Data Ingestion:** The Python script running on the ROS2 robot periodically sends HTTP POST requests containing JSON telemetry and Base64 images to /api/robot/telemetry and /api/robot/alert.

2. **Data Persistence & Processing:** Spring Boot maps this JSON to DTOs, evaluates business logic (e.g., triggering a "LOW_BATTERY" internal alert if the battery drops below 15%), and saves the entities to MySQL.

3. **WebSocket Broadcast:** Simultaneously, using the SimpMessagingTemplate, the backend broadcasts the newly saved entities directly to the STOMP message broker topics.

4. **Frontend Reactivity:** The React component's useEffect hooks, which are actively listening to these topics, receive the new payload and call setState, triggering an immediate re-render of the live metrics on the screen.